DOCUMENT 2:
QUICK-START GUIDE (How to Install DTAF)

This document outlines the procedures for installing the DTAF AI Pipeline. It serves as the starting point for deploying DTAF by presenting step-by-step configuration requirements and installation instructions. This guide is organized into three sections: Section 1 describes how to prepare the AI pipeline environment; Section 2 provides installation instructions for the required software components; and Section 3 provides information on future DTAF model updates. The full installation typically takes 20-40 minutes, depending on hardware and network speed.

1. Preparing the AI Pipeline Environment

Certain software applications are necessary to execute the DTAF AI pipeline. Table 1 indicates which ones are required and offers alternative options.

Table 1: Recommended Software Requirements.

Requirements	Recommended	Alternatives
Python	Python	None
Code Editor	VS Code	Yes (Any Python Integrated Development Environment)
AI Model	Gemma3:12b	Yes (See Table 2)

Note: This installation guide document assumes the user is utilizing the recommended software and operating on a Windows system.

To support privacy and secure deployment, the pipeline utilizes a Large Language Model (LLM) running locally on the user's machine. Local execution is particularly important for manufacturing organizations that may need to discuss sensitive operational data, proprietary workflows, production bottlenecks, or strategic priorities. Because the model runs locally, organizational information does not need to be transmitted to external cloud-based AI services, reducing concerns regarding data leakage, confidentiality, or unauthorized reuse of proprietary information.

The DTAF AI pipeline is set up to recommend an AI model based on the user's individual hardware specifications. Gemma3:12b is used as the recommended default because it provides a practical balance between reasoning capability and local hardware requirements. However, alternative models such as LLaMA, Mistral, Phi, or smaller Gemma models may also be used when appropriate. This flexibility is important because some organizations may prefer model families that are more familiar within defense, engineering, or research environments while others may prefer smaller models for lower-resource computers. Example of model options are provided in Table 2.

Table 2: Model Options and Uses.

Model Option	Suggested Use
Phi-3.5:3.8B	Recommended low-memory model for systems with limited RAM, VRAM, or CPU resources
Gemma 2:2B	Very lightweight alternative for constrained systems, quick testing, and basic functionality verification
Gemma 3:12B	Recommended default model for balanced systems, prioritizing structured-output quality on systems.
LLaMA 3.1:8B	Faster general-purpose alternative for balanced systems and organizations familiar with Meta LLaMA models
Mistral Nemo:12B	Higher-quality alternative for balanced or high-performance systems where some additional inference time is acceptable
Mistral 7B Instruct v0.3 Q4_K_M	Quantized, memory-efficient alternative offering good instruction-following performance on systems with limited VRAM
Gemma 2:9B	Higher-quality model option for systems with additional RAM or VRAM
LLaMA 3.1:70B	Workstation-class model for systems with substantial RAM, VRAM, or combined CPU and GPU inference capacity
Gemma 2:27B	Workstation-class alternative for high-capacity systems where output quality is prioritized over inference speed

Note on AI Model Alternatives: The DTAF AI Pipeline will recommend an AI model depending on the user's individual hardware specifications. Model performance may vary by computer. Before formal use, the selected model should be tested to confirm that it can complete the DTAF AI pipeline, follow formatting requirements, and generate usable structured outputs. To achieve reliable performance when running the Gemma3:12b pipeline locally, the following minimum and recommended hardware specifications are suggested in Table 3 below.

Table 3: Recommended Computer Requirements.

Component	Minimum Tested	Maximum / Recommended Tested
RAM	32 GB	64 GB
CPU	Intel Core Ultra 5 235H, 14 cores / 14 threads	AMD Ryzen 9 7950X, 16 cores / 32 threads
VRAM	8 GB	16 GB
Storage	30+ GB SSD	30+ GB NVMe SSD

2. Installation Instructions

STEP 1: Install Python: Python is required to run the pipeline.

- Visit the official Python website: https://www.python.org/downloads/
- Download the latest version.
- Run the installer and complete the installation.

STEP 2: Install Visual Studio Code (Recommended): The DTAF AI pipeline uses Visual Studio Code (VS Code), although any Python interpreter may be used.

- Download VS Code: https://code.visualstudio.com/
- Install the software.
- Open VS Code.
- Go to the Extensions icon on the left sidebar, search for "Python," and click Install.

STEP 3: Install Ollama: Ollama allows the AI model to run locally on your computer to protect sensitive company data because information is not sent to external cloud servers.

- Visit: https://ollama.com/
- Download Ollama.
- Install the software.
- Open Ollama once after installation.

STEP 4: Install Gemma 3:12B (Recommended): The DTAF pipeline uses Gemma 3:12B. This model was selected because it provides strong reasoning performance while remaining manageable on standard hardware. Note: When first run, the DTAF pipeline will recommend an AI model suited to the computer's hardware, which may differ from Gemma3:12b; installing Gemma3:12b as described here provides a reliable default.

- Open Command Prompt on your system by pressing the ***Windows key + R.***
- Type ***cmd*** and press ***Enter.***
- Type **ollama pull** followed by the preferred model tag from Table 2. For example, if using the recommended Gemma3:12b model, copy and paste the following command into the terminal and press Enter: **ollama pull gemma3:12b**.
    - Additionally, to download all supported DTAF models, run the following PowerShell command (this will take a long time to download all models):
        - "phi3.5:3.8b","gemma2:2b","gemma3:12b","llama3.1:8b","mistral-nemo:12b","mistral:7b-instruct-v0.3-q4_K_M","gemma2:9b","llama3.1:70b","gemma2:27b" | ForEach-Object { ollama pull $_ }
- Wait until the installation is completed.

STEP 5: DTAF AI GitHub: The files and materials needed to run the DTAF AI pipeline are within the GitHub repository.

- Visit the GitHub repository that contains the DTAF AI pipeline files: https://github.com/lkw0012-sudo/AI-DTAF/tree/main
- Click the provided project package named ***DTAF.zip*** (Figure 1) and download the zip folder (Figure 2).

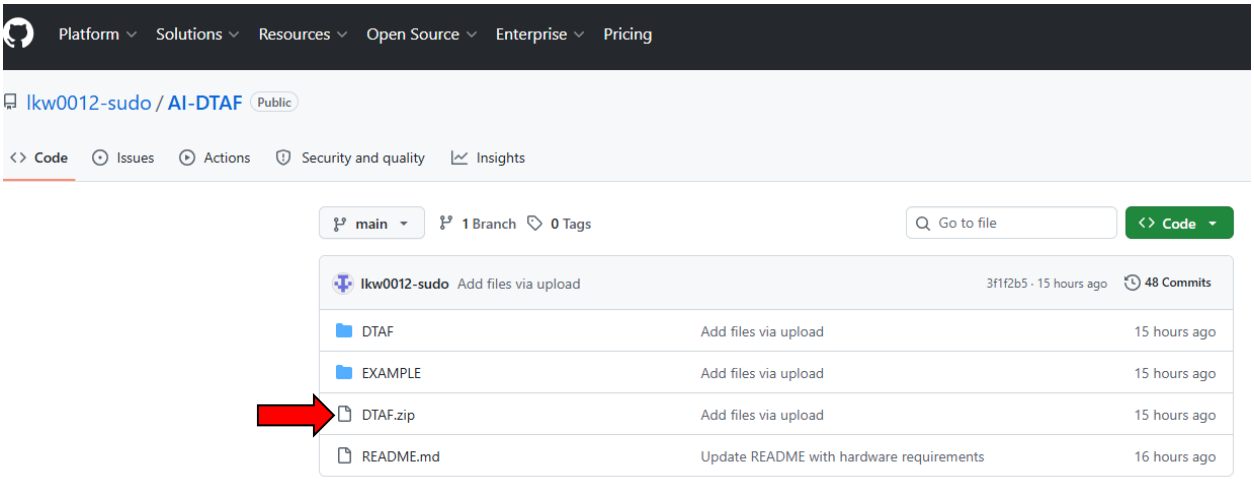


Figure 1: Locating the DTAF.zip folder from GitHub.

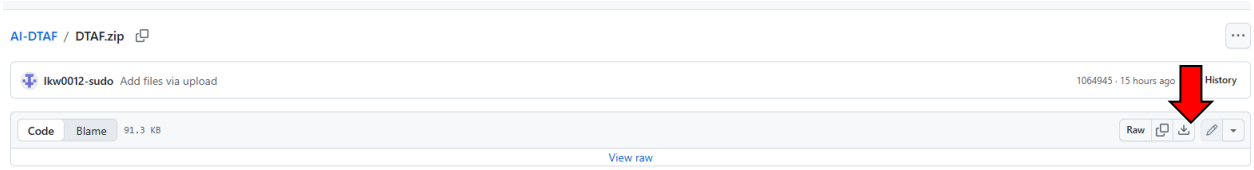


Figure 2: Downloading the DTAF.zip folder from GitHub.

- Go to the Downloads folder, right-click the ZIP file, and select Extract All.
- When prompted, choose a permanent location for the extracted folder, such as Documents or the Desktop, rather than running the files from inside the ZIP or the Downloads folder. This makes the program easy to find and rerun after it is closed.
- Open the extracted folder. This is where all the required files are located.
    - *Important:* Do NOT remove files from the extracted folder. All files must remain together. If the main Python file is moved elsewhere, the program may fail because it depends on internal folders and knowledge-based files.
- Inside the extracted folder (Figure 3), identify the file **DTAF.py**. This is the main executable file the user will run.
    - Note that the File order may appear different for each user. Other files in this folder are:
        - JSON Files: These serve as the AI knowledge base and contain logic for the DTAF stages, capability mappings, decision rules, and framework guidance.

- Markdown (.MD) Files: These contain reference material from the Digital Twin Consortium, including capability descriptions, prompt guidance, and capability Periodic Table definitions.

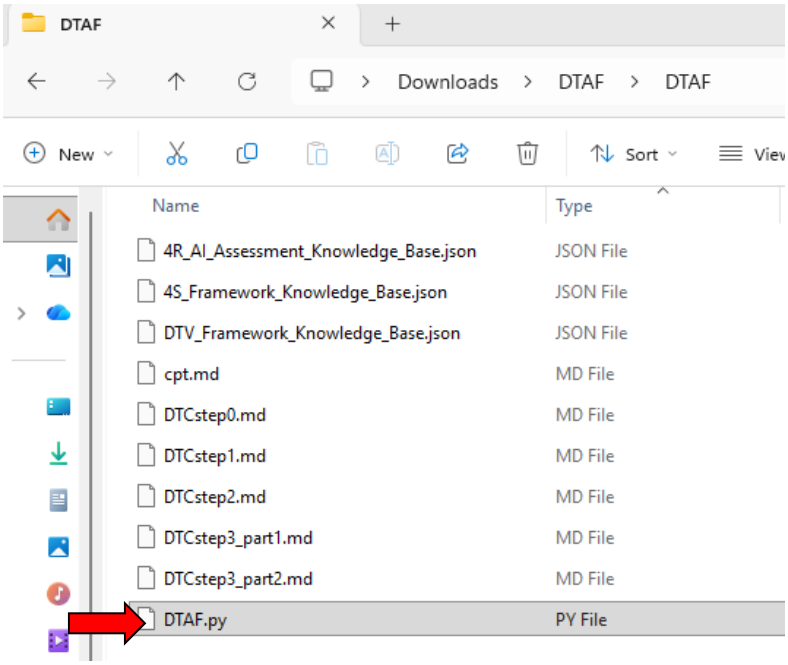


Figure 3: Extracted DTAF.zip folder.

STEP 6: Running the Program:

- In the extracted folder, double-click on DTAF.py to run the Python file within VS Code.
- The Python file will look like the following (Figure 4) when opened with VS Code.
- Click the Run button identified with the red arrow in Figure 4.
- The AI pipeline will start in the terminal at the bottom of VS Code.
- The installation is complete. The next steps on how to navigate and answer the questions through the AI Pipeline are provided in **Document 3: Full DTAF Case Study**.

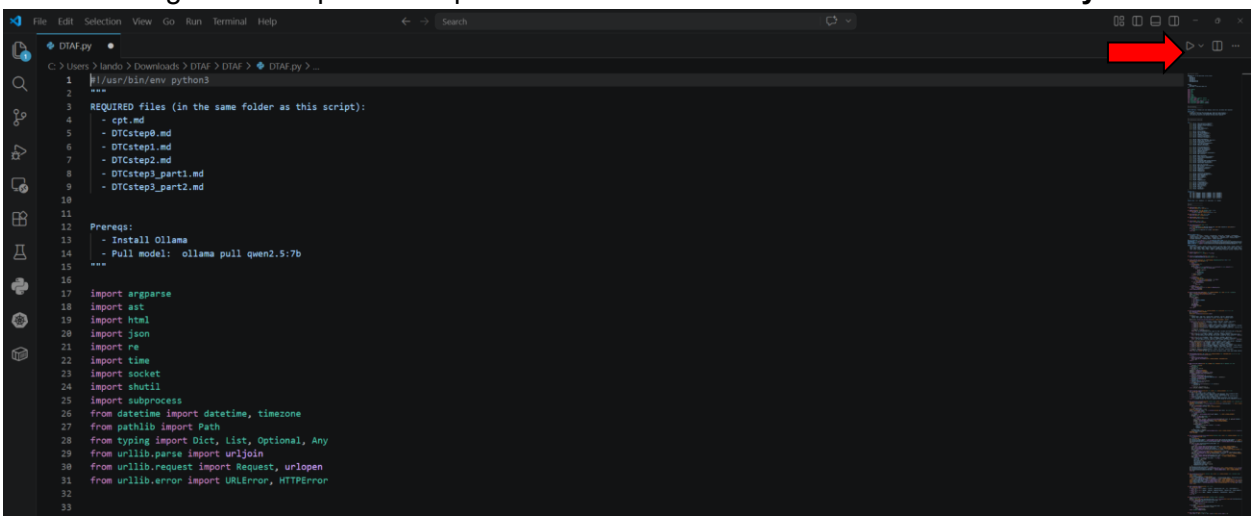


Figure 4: Python file opened in VS Code.

3. DTAF Instructions and Model Updates

The DTAF AI pipeline will be maintained through the project's GitHub repository. As the tool continues to develop, updates may be made to the Python code, installation instructions, knowledge-based files, and AI model options. These updates will be posted on GitHub to help ensure that the DTAF remains usable as software versions, local LLM options, and DT assessment needs change over time. The repository is created and maintained by the DTAF research team, which retains ownership of the pipeline code and documentation; updates are published by the team, and users can report issues or request changes through the repository or the feedback form below. Major changes will be documented using version updates located within the GitHub space (as pointed to with a red arrow in Figure 5) or brief release notes so users understand what has changed from one version to the next.

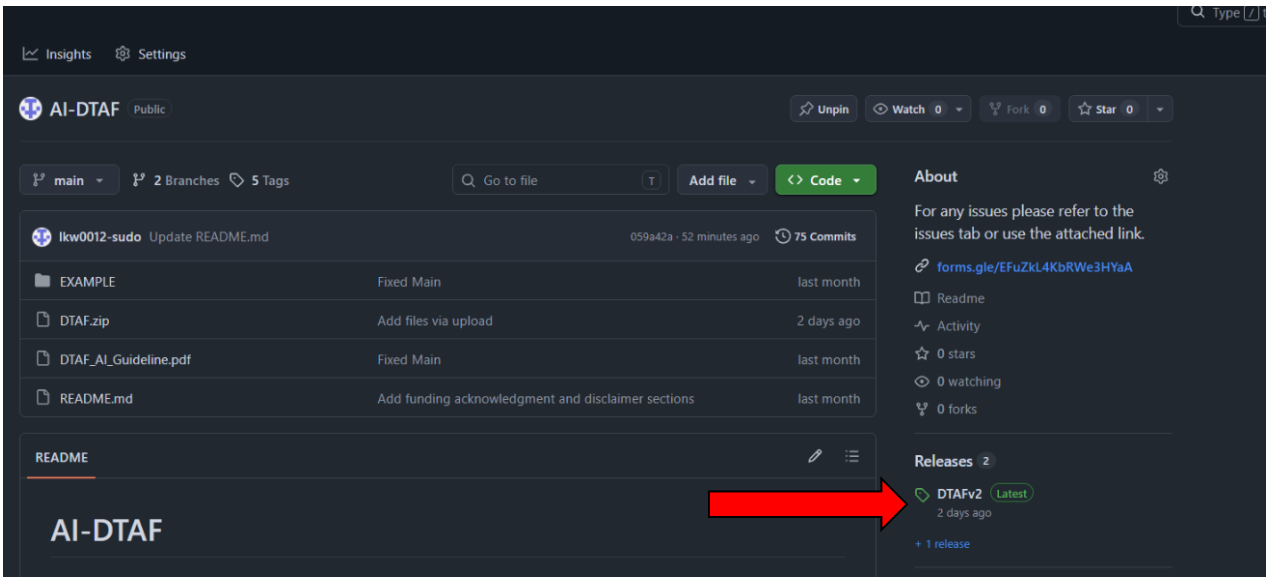


Figure 5: Where to find the newest release.

Future updates may include support for additional local AI models, improvements to output formatting, bug fixes, revised documentation, and updates to the CNC case study example. If you have any additional comments, suggestions, or encounter issues that are not covered in this guide, please submit your feedback using our Google Form.

This form was created to collect user experiences with the DTAF AI Pipeline, including feedback on the installation process, documentation, usability, and any technical issues. Your responses will help us identify areas for improvement and make the pipeline easier to install and use for future users. While Gemma3:12b is currently used as the recommended model, additional models such as LLaMA, Mistral, Phi, or other Ollama-supported models may be tested and added as alternatives.

To maintain consistency, the DTAF folder structure should remain intact, and the CNC demonstration (or other existing example) should be rerun after major updates to confirm that the pipeline still works correctly. This will help keep the DTAF AI Pipeline easy to install, easy to run, and usable for future DT assessment work.